

COMPUTER SCIENCE TRIPOS Part IB – 2021 – Paper 4

3 Compiler Construction (tgg22)

Optimisation

- (a) Suppose we are writing a compiler for an ML-like language and we want to employ the equation

$$(\text{map } f \text{ } l1) @ (\text{map } f \text{ } l2) = \text{map } f (l1 @ l2)$$

as a left-to-right rewrite rule for optimisation. The symbol $@$ represents list append.

Discuss the merits of this idea. Is it always correct? If so, state clearly what assumptions you are making about $@$ and map . [5 marks]

Answer: It is not at all clear that this optimisation is worthwhile. The function f has to be applied to the same number of arguments on each side of the equation and $@$ is applied to lists of the same size. So this does not seem like a useful optimisation (2 marks). However, it does seem correct even if function f has side-effects. That will be true as long as $@$ evaluates its left argument first and map processes lists from head to tail **or** $@$ evaluates its right argument first and map processes lists from tail to head.

Compiler
front-end

- (b) A compiler's front-end will often expand some syntactic constructs into lower-level representations. Consider the following fragment for the abstract syntax of a SLANG-like language.

```
type var = string

type exp =
  (* abstract syntax *)      (* concrete syntax *)
  | Var of var               (* x *)
  | Project of int * exp     (* proj i e *)
  | Tuple of exp list        (* (e1, e2, ..., en) *)
  | Let of var * exp * exp   (* let x = e1 in e2 *)
  | Apply of exp * exp       (* e1 e2 *)
  | Function of var * arg_pattern * exp (* fun f p = e *)

and arg_pattern =
  | APvar of var             (* x *)
  | APtuple of arg_pattern list (* (p1, p2, ... pn) *)
```

This language has general projections for n -tuples so

$$\text{proj } i (e_1, e_2, \dots, e_k)$$

will evaluate to v_i , the value of e_i . If i is not in the range between 1 and k there will be a run-time error.

In this language we can write functions with simple (possibly nested) patterns for function arguments:

```
fun f (a, b, (c, (d, e))) = b a
```

Now suppose we want our front-end to eliminate such patterns. That is, we want to write a function of type

```
eliminate_tuple_patterns : exp -> exp
```

so that the resulting expression contains functions with patterns only of the form `APvar x` for some (new) variable `x`. For example, the code for `f` above should be translated to a semantically equivalent expression of the form

```
fun f x = ...
```

that contains only simple variable arguments (that is, only `APvar` patterns in the abstract syntax).

Your task is to write this function in OCaml. You can assume that you have a function for generating fresh variable strings.

```
new_var : unit -> string
```

[15 marks]

Answer: There are many possibilities. One approach is to turn patterns into let bindings. For example, suppose we have the following code.

```
fun f (a, b, (c, (d, e))) = b a
```

This should be translated to something like this:

```
fun f x =  
  let a = proj 1 x in  
  let b = proj 2 x in  
  let c = proj 1 (proj 3 x) in  
  let d = proj 1 (proj 2 (proj 3 x)) in  
  let e = proj 2 (proj 2 (proj 3 x)) in b a
```

I have attempted to make my code as clean as possible:

```
let new_var =  
  let i = ref 0 in  
  let get () = let v = !i in (i := (!i) + 1; "_x" ^ (string_of_int v))  
  in get
```

```
type var = string
```

```
type exp =  
  | Var of var                                (* x *)  
  | Project of int * exp                      (* proj i e *)  
  | Tuple of exp list                        (* (e1, e2, ... en) *)  
  | Let of var * exp * exp                   (* let x = e1 in e2 *)  
  | Apply of exp * exp                       (* e1 e2 *)  
  | Function of var * arg_pattern * exp      (* f p = e *)
```

```
and arg_pattern =
  | APvar of var                (* x *)
  | APtuple of arg_pattern list (* (p1, p2, ... pn) *)

(*
This is a good use of mapi!

val mapi : (int -> 'a -> 'b) -> 'a list -> 'b list
  Same as List.map, but the function is applied to the index of
  the element as first argument (counting from 0), and the element
  itself as second argument.
*)

let make_lets_from_pattern x body p =
  let rec pattern_to_bindings g = function
    | APvar y      -> [(y, g())]
    | APtuple pl ->
      List.concat (
        List.mapi (fun i ->
          (pattern_to_bindings
            (fun () -> Project(i+1, g())))) pl)
      in let rec make_lets = function
        | [] -> body
        | (v, e)::rest -> Let(v, e, make_lets rest)
      in make_lets (pattern_to_bindings (fun () -> Var x) p)

let rec eliminate_tuple_patterns = function
  | Var x                ->
    Var x
  | Project (i, e)       ->
    Project(i, eliminate_tuple_patterns e)
  | Tuple e1             ->
    Tuple (List.map eliminate_tuple_patterns e1)
  | Let(x, e1, e2)       ->
    Let(x, eliminate_tuple_patterns e1,
    eliminate_tuple_patterns e2)
  | Apply(e1, e2)        ->
    Apply(eliminate_tuple_patterns e1,
    eliminate_tuple_patterns e2)
  | Function(f, APvar x, e) ->
    Function(f, APvar x, eliminate_tuple_patterns e)
  | Function(f, p, e)     ->
    let x = new_var () in
    let e' = eliminate_tuple_patterns e in
    Function(f, APvar x, make_lets_from_pattern x e' p)
```

Indeed, this works as desired for the example presented above.

```
let e = Function("f",
  APtuple [APvar "a"; APvar "b";
    APtuple[APvar "c";
      APtuple [APvar "d"; APvar "e"]]],
  Apply(Var "b", Var "a"))
```

```
let test = eliminate_tuple_patterns e

val test : exp =
  Function ("f", APvar "_x0",
    Let ("a", Project (1, Var "_x0"),
      Let ("b", Project (2, Var "_x0"),
        Let ("c", Project (1, Project (3, Var "_x0")),
          Let ("d", Project (1, Project (2, Project (3, Var "_x0"))),
            Let ("e", Project (2, Project (2, Project (3, Var "_x0"))),
              Apply (Var "b", Var "a"))))))))
```

This matches closely the code above:

```
fun f x =
  let a = proj 1 x in
  let b = proj 2 x in
  let c = proj 1 (proj 3 x) in
  let d = proj 1 (proj 2 (proj 3 x)) in
  let e = proj 2 (proj 2 (proj 3 x)) in b a
```

Note that there is still room for improvement. For example, common sub-expressions could be removed:

```
fun f x =
  let a = proj 1 x in
  let b = proj 2 x in
  let z1 = proj 3 x in
  let c = proj 1 z in
  let z2 = proj 2 z1 in
  let d = proj 1 z2 in
  let e = proj 2 z2 in in b a
```

Another approach might be to perform substitutions rather than using lets. For example,

```
fun f (a, b (c, (d, e))) = b a
```

could become

```
fun f x = (proj 2 x) (proj 1 x)
```
